Evaluating the Performance of Java Application Caches in Web and Database-Driven Applications

Vasco Senra[1[0009−0005−7756−129X]], Paul Crocker[1[0000−0001−6824−6136]], and Valderi Leithardt[2[0000−0003−0446−9271]]

[1] Institute of Telecommunications & University of Beira Interior, Covilhã, Portugal {vasco.senra,crocker}@ubi.pt
[2] Lisbon University Institute (ISCTE-IUL), ISTAR, Lisbon, Portugal valderi.leithardt@iscte-iul.pt

Abstract. Modern web applications face growing challenges in latency, resource usage, and scalability, particularly under high user concurrency. This study evaluates caching strategies for Java-based REST API applications that access data repositories, focusing on both in-memory and distributed in-memory solutions. The approaches are compared using raw performance measurements and a novel evaluation metric to assess their impact on system performance, scalability, and resource efficiency. Results show that no single caching solution is universally superior; instead, each presents distinct trade-offs depending on workload and deployment context. These findings provide practical guidance for developers seeking to optimize response times, reduce system load, and improve overall user experience.

Keywords: cache,efficiency,Java,latency,optimization,performance,resource management,scalability,web applications

1 Introduction

The growth of the Internet has led to an enormous volume of data and increasingly intensive concurrent access. This reality imposes significant challenges on modern applications, which must handle longer response times, higher resource consumption, and the need to maintain consistent performance even under high demand. Applications that fail to address these challenges are likely to suffer from performance degradation, system overload, and have a negative impact on user experience. As a result, researching strategies such as caches that improve operational efficiency and enhance the scalability of complex applications has become essential. The term cache derives from the French language, deriving from the verb *cacher*, which means "to hide" or "to keep" [1]. A cache is a fast and temporary data storage mechanism that aims to improve the performance of a system through the preservation of copies of frequently accessed information.

This article presents an analysis of different caching approaches in Java REST API applications for data repositories that store, manage, share and preserve

research datasets in particular those that have been developed for the communities in the EPOS project [2]. The selected cache technologies are evaluated in terms of their impact on performance and scalability. The final goal is to provide practical insights for developers and system architects who seek to maintain high-performance applications under demanding conditions.

The remainder of this article is organized as follows. After the introduction a brief review of Cache Technologies and State of the Art is given where relevant literature is discussed and research gaps that motivate this investigation are identified. This is followed by a description of the methodology employed, the system Architecture and Metrics that are used to test the selected caches. The experimental results section provides an analysis of the cache performance results obtained, including a comparative evaluation of the distinctive features observed across the different cache implementations. Finally, we offer concluding remarks that underscore the relevance of this research, summarize its main contributions, and outline potential directions for future work.

2 Cache Technologies and State of the Art

2.1 Cache Principles

A cache typically operates through three core mechanisms: *write*, *read*, and *validation*. In the *write* phase, when data is requested for the first time and is not present in the cache (a cache miss), the system retrieves it from the authoritative data source. The retrieved value is then stored in the cache to optimize subsequent access. In the *read* phase, upon receiving a request, the system checks whether the corresponding entry exists in the cache (a cache lookup). If the entry is present it is first validated i.e that the cached entry is still correct, unexpired, and consistent with the original source in which case it is returned immediately. If the entry is absent, expired, or invalid, the system fetches the latest data from the original source and updates the cache accordingly.

The use of a cache offers several advantages. First, it may decrease response time by avoiding repeated and burdensome operations. Caching also reduces the load on the original system as local storage of data copies lowers the number of queries to the database or remote services. However, caching also has some limitations. The stored information may become outdated, which can compromise result consistency. In addition, configuring the cache can be complex. Finally, excessive use or poor configuration of the cache may negatively impact performance, increase memory consumption and processing overhead.

2.2 Literature Review

Several studies have analyzed cache strategies and their application in web-based systems; however, many of them present limitations related to application context and research scope.

Kabakus (2020) [3] compared different cache implementations and demonstrated that performance largely depends on architectural design choices; nevertheless, the study relied on outdated libraries and simplified experimental scenarios. Hasslinger (2023) [4] reviewed evaluation methods for cache replacement policies, emphasizing the importance of using multiple performance metrics, but provided limited practical application within the Java ecosystem. Noetzold(2024) [5] demonstrated that the JVM configuration strongly influences system performance. Kanthed(2023) [6] compared Redis and Memcached in microservices, reinforcing the relevance of storage and replication strategies in distributed scenarios. Henckel (2022) [7] showed that in-memory caching techniques, together with database synchronization mechanisms, significantly impact both latency and data consistency. Zulfa (2020) [8] systematically reviewed web caching, prefetching, and application-level caching strategies, emphasizing differences in replacement policies and their effect on consistency guarantees. Gupta et al.(2021) [9] evaluated the performance of several in-network caching strategies in Information-Centric Networking using two performance metrics: Cache Hit Ratio (CHR) and Content Retrieval Delay (CRD). CHR measures the proportion of content requests served directly from in-network caches, reflecting cache utilization efficiency and traffic offloading, while CRD measures the total time required to retrieve content, representing user-perceived latency.

Overall, the existing literature exhibits significant gaps and remains relatively sparse. Many studies are either theoretical, outdated, or restricted to narrowly defined contexts, which limits the ability to perform comprehensive and up-to-date comparisons between caching frameworks in Java.

2.3 Cache libraries for Java

The caching solutions considered in this work were selected based on their suitability for Java-based environments, specifically those running on the JVM and preferably embedded within the application code. In this context, Apache Ignite was included as an example of a distributed caching technology due to its origin and design focus on the Java ecosystem, as it was initially developed for the JVM and provides native integration within Java applications. Conversely, technologies such as Redis [10] were excluded from the analysis, as they follow a more generic and language-agnostic adoption model, making them more universal rather than tightly coupled to Java environments. The cache libraries for Java selected in this study incorporate several architectures and functionalities, which will be described in the following sections.

- Guava [11] provides local memory cache, configurable by size, TTL, and removal policies, with support for weak and soft references, ensuring thread safety.
- Apache Ignite [12] is a distributed in memory platform, combining in memory and disk storage , with parallel processing and consistency made sure by the Two-Phase Commit. The current version 3 lacks detailed documentation unlike the previous versions which difficult the development process.

- Ehcache [13] allows in memory cache, outside the heap or on disk, supporting cache hierarchies and programmatic configuration.
- Caffeine [14] offers high performance inspired by Guava with advanced cache policies, flexible configuration and detailed monitoring of metrics.

All the features discussed are current as of today and correspond to the most recent version of the caches available and are summarized in table 1. Note that some features may vary in earlier or future versions. The complete version list referenced in this study is available in the repository [15].

Table 1: Cache Software Characteristics

Features/Cache	**Ehcache**	**Caffeine**	**Guava**	**Ignite v3.0**
Storage	Heap,off-Heap,Disk	Heap	Heap	Distributed RAM
Persistence	Yes	No	No	Yes
Cache MISS	No	Yes	Yes	Yes
Cache Size	MB	Entries/MB	Entries/MB	Max 64kb entry
TTL	Yes	Yes	Yes	No
Eviction algorithms	LRU, LFU, FIFO	W-Tiny LRU	LRU	None
Implementation	Medium	Simple	Simple	Complex
Documentation	Good	Decent	Decent	Very Basic

3 Methodology, System Architecture and Metrics

3.1 Methodology

This study adopted an experimental approach to evaluate the impact of different cache mechanisms on API performance. The methodology involved developing execution environment scenarios, implementing of the specific cache solutions, performance testing, and analysis of specific metrics. Aligned with the FAIR principles, this study provides all data and source code, enabling other researchers to reuse the materials and compare their findings with the results presented here [15].

The environment was structured to guarantee reproducibility and consistency in testing. A database ran in a Docker container whilst the API ran natively in the host, with controlled resources. A base version of the API (API-BASE), without cache mechanisms, was created, which served as a reference to the initial tests and to define performance metrics.

Several performance metrics were monitored: CPU usage, RAM memory consumption, disk performance (IOPS), response quality and quantity of valid responses. When correlated with the average response time, these metrics enabled the evaluation of the impact of cache mechanisms on the API's overall performance.

Furthermore, a thousand calls to the API were made at different levels of concurrency, ranging from ten to one hundred simultaneous calls, maintaining a constant number of parallel executions in each scenario. The objective was

to identify appropriate levels of workload and measure the difference in system behavior.

Each of the selected cache mechanisms was implemented in a distinct version of the API according to the following nomenclature: "API-cache system". Consistent tests simulated multiple equivalent requests, while keeping constant the quantity, complexity and frequency of the requests, thereby allowing objective comparison between versions.

The metrics obtained for the different cache mechanisms were then analyzed to identify bottlenecks, resource efficiency and unexpected behaviors.

3.2 Test System Architecture

The system was developed based on the three-tier architecture. The data tier uses a container Docker with PostgreSQL 15 for persistent storage. The API, developed using the Spring Framework in IntelliJ, manages endpoints, performs queries SQL and generates CSV files with performance metrics. Additionally, the client tier uses Python scripts to simulate requests and test the system on different workloads. The following figure 1 represents the adopted architecture in the system.

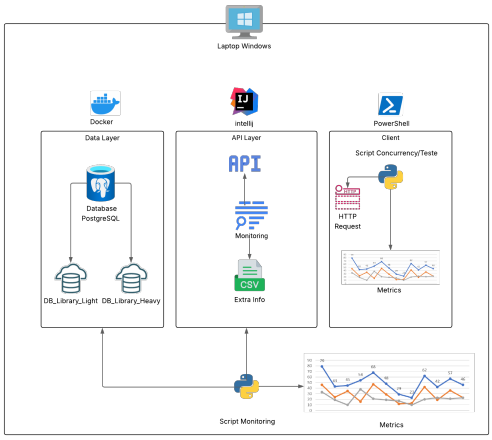

Fig. 1: The Test System Architecture.

The tests in this paper were conducted on a Lenovo Legion 5 (i7-14650HX, 16 GB RAM, NVMe SSD). Four cores and four GB of ram were allocated to the Docker and four cores and eight GB of ram to the API, the client tier didn't receive dedicated resources. This distribution follows microservices practices with isolated data bases and APIs with higher CPU and memory priority [16].

3.3 Data Sets

Two datasets for the database - "Light" and "Heavy" - were developed with an identical schema whilst containing distinct volumes of information. The data

model was intentionally designed to be inefficient by incorporating excessively long fields, intensive use of the `TEXT` type, `%ILIKE%` searches, and deliberately inadequate modeling options to increase computational complexity.

The data was generated using Python and with the `Faker` [17] library. The "Heavy" version, contains substantially larger texts. This means that queries generate significant memory consumption and thus queries are limited to a minimal of four words search to avoid excessive timeouts and API overload.

This article addresses only the tests conducted on the heavy-database due to space limitations. However, all experiments and conclusions are fully documented in my thesis and repository [15].

3.4 API-Base

The REST API developed in this study, designated "API-Base", constitutes the fundamental version on which the different cache mechanisms were subsequently integrated. It was implemented in Spring Boot and conceived exclusively for GET operations. Thus, it works as a data query system without insertions, alterations, or deletions.

The structure is organized into simple and composite endpoints, varying the complexity according to the number of relationships between tables.

The application developed follows the typical Model-View-Controller (MVC) design pattern, where the "Controller" receives and validates the request, forwarding it to the Service layer that interacts with the Spring Data JPA repositories (Model layer). The results coming from the database are converted into Java objects and returned to the "Controller", where they are parsed and formatted to a return type, JSON or CSV, before being sent to the client. The API implements basic mechanisms of validation, returning appropriate HTTP codes depending on the request status and possible timeouts. This application is represented in the following dataflow 2.

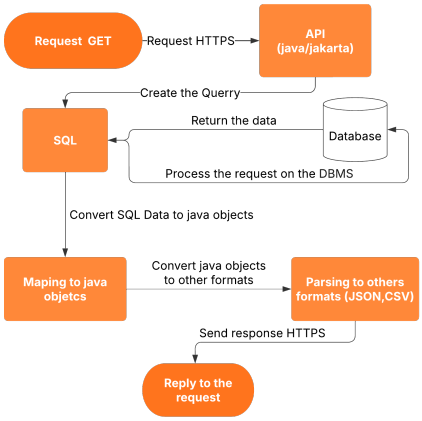

Fig. 2: Application Dataflow.

3.5 ICE Metric

To measure the efficiency of each cache, we propose the following weighted **ICE** (Indicator of Cache Efficiency) metric, which measures how a cache improves or degrades execution time and hit rate relative to a baseline. The metric is defined as:

$$\text{ICE} = (k_3 \cdot T_{\text{imp}}) + (k_4 \cdot H_{\text{imp}}) - (k_1 \cdot T_{\text{deg}}) - (k_2 \cdot H_{\text{deg}}) \tag{1}$$

where:

$$\begin{aligned} T_{\text{imp}} &= \max(0, -T_d), && \text{(time improvement \%)} \\ T_{\text{deg}} &= \max(0,\ T_d), && \text{(time degradation \%)} \\ H_{\text{imp}} &= \max(0,\ H_d), && \text{(hit-rate improvement \%)} \\ H_{\text{deg}} &= \max(0, -H_d), && \text{(hit-rate degradation \%)} \end{aligned}$$

and $k_1, k_2, k_3, k_4 \geq 0$ are (relative weights) that allow developers to emphasize or penalize specific parameters as needed. For example, in systems where correctness is critical, one may assign a higher weight to k_2 and reduce k_1, prioritizing error-related metrics over execution time. However, a high ICE value does not imply that one cache is universally better. It only indicates superior performance under the chosen conditions and weight settings.

4 Experimental Results

In this section, the collected performance data is presented and analyzed. Several challenges emerged regarding how to represent the results. Many metrics showed small absolute differences or low variance, but their relative (percentage-based) variations were more expressive and therefore required careful treatment. Another difficulty stemmed from the inherently multivariate nature of the dataset.

The next graph Figure 3 shows the hit rate by cache type and concurrency level, allowing a comparison of the efficiency between cold and operational modes and concurrency. The bars represent the absolute value of hits obtained, while the end of each bar indicates the percentage change relative to the base API.

Figure 4 below shows, in two dimensions, the relationship between: (i) total execution time in minutes (x-axis, with lower values being preferable) and (ii) number of hits (y-axis, with higher values being desirable). Additional information was included, such as the level of concurrency and the operating status: cold, corresponding to the system with an empty cache and in the process of initialization, and operational, corresponding to the system with a full cache and in a stable state. All these variations are comparable to the base API.

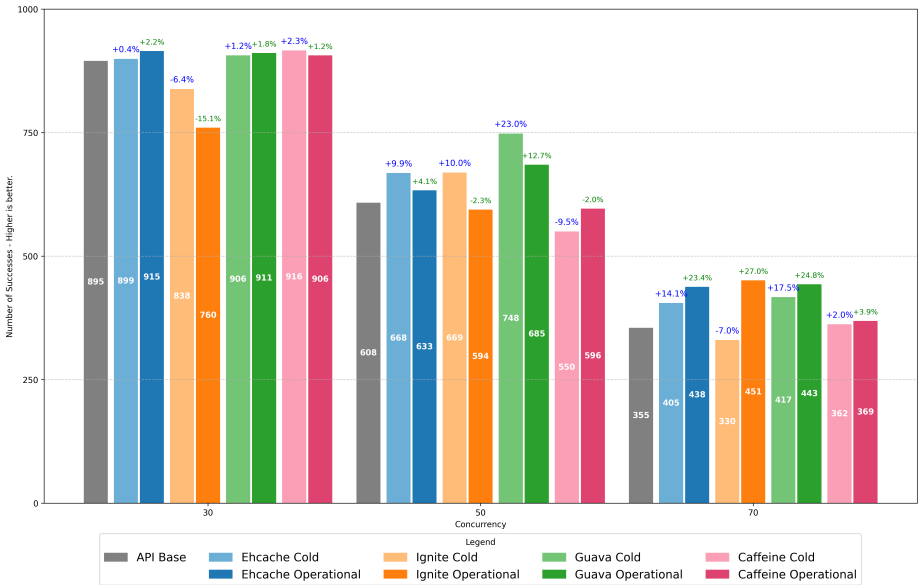

Fig. 3: Hit rate by cache,concurrency and operational mode.

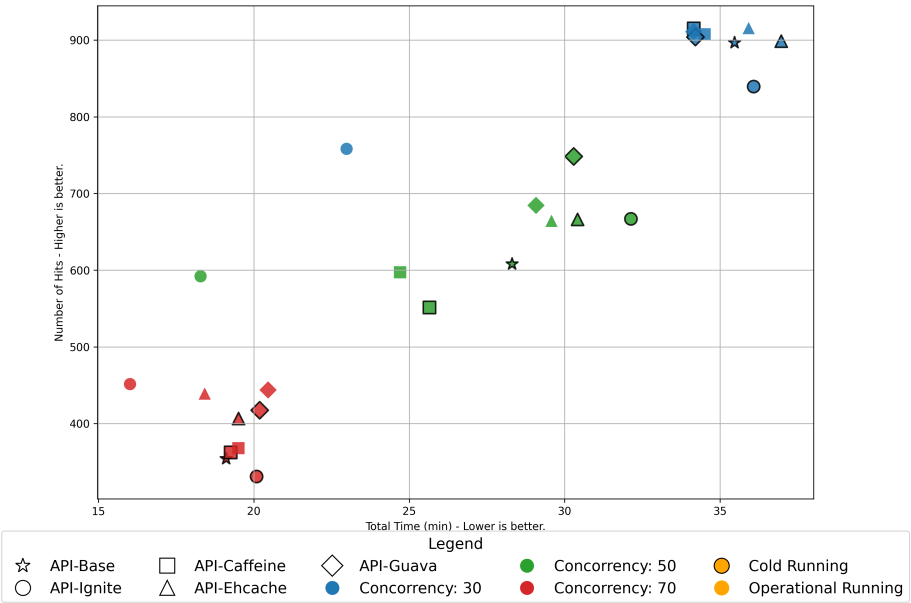

Fig. 4: Scatter Plot by Categories.

Finally, table 2 shows the resources used by each cache system. All values are normalized relative to the base API to facilitate comparison between configurations. However, the base API also includes absolute values, thus allowing direct conversion between normalized and actual measurements.

Table 2: Performance results for API and Database.

API

Cache	Avg CPU	Max CPU	Avg RAM	Max RAM
Base	1.8 %	68.3 %	3.80 GB	6.40 GB
Caffeine	0.889	0.676	1.526	1.047
Ehcache	3.556	0.836	1.447	1.172
Guava	0.778	0.717	1.211	0.906
Ignite*	7.611	2.097	1.487	2.029

Database

Cache	Avg CPU	Max CPU	Avg RAM	Max RAM
Base	95.2 %	107.8 %	3.14 GB	3.22 GB
Caffeine	1.047	1.019	1.003	1.000
Ehcache	1.022	1.020	0.981	1.016
Guava	1.054	1.026	1.013	1.003
Ignite	0.996	1.007	0.987	0.997

*Ignite includes an additional distributed processing component; all CPU and memory resources associated with this component were included in the totals.

Analysis of the resource consumption results reveals significant differences between the evaluated cache systems.

At the API layer, normalized values show that Ignite has an average CPU consumption 7.6 times higher than the base API, while Caffeine and Guava caches maintain consumption at levels below the base, at 0.89 and 0.78, respectively. Ehcache has an intermediate value (3.56), indicating considerable effort, but still below Ignite. Ignite's maximum CPU consumption is also higher, reaching 2.1 times the base value, whilst the other caches remain below 1, which shows that Ignite imposes a high cost on the CPU due to distributed processing and components.

In terms of memory usage, all caches consume more RAM on average than the base API, which is expected due to the storage of information in memory. The average values range from 1.211 for Guava, which occupied the least space, to 1.526 for Caffeine. In terms of maximum consumption, Ignite showed the highest value at 2.029, while Guava was the most efficient at 0.9, being the only cache with a maximum consumption lower than the base API.

When analyzing the impact on the database, it can be observed that Ignite is the only cache that manages to slightly reduce the average CPU consumption of the database to 0.996 of the base, and also shows a slight reduction in average RAM usage (0.987), suggesting that it effectively contributes to alleviating the load on the database. Caffeine and Guava, on the other hand, show a marginal increase in the database's CPU consumption (1.047 and 1.054), indicating that, although efficient in the API, their effect on reducing the load on the database is less significant. Ehcache maintains values close to 1 for CPU and RAM, reflecting intermediate behavior, with a neutral impact on the database.

4.1 Indicator of Cache Efficiency Metric

In this section, we use the formula in section 3.5 to calculate the ICE metric using specific weights for two specific cases:

$$\text{Weights a)}(1 \cdot T_{\text{imp}}) + (2 \cdot H_{\text{imp}}) - (1.1 \cdot T_{\text{deg}}) - (2.2 \cdot H_{\text{deg}})$$
$$\text{Weights b)}(2 \cdot T_{\text{imp}}) + (1 \cdot H_{\text{imp}}) - (2.2 \cdot T_{\text{deg}}) - (1.5 \cdot H_{\text{deg}})$$

The first system of weights focuses on improving response quality, in which correct answers are valued more highly and errors are penalized. The time factor is rewarded, but it carries limited weight when performance improves, and a slightly higher weight when performance deteriorates. The second set of weights is the opposite: it prioritizes fast responses and penalizes slower ones, while moderately penalizing incorrect answers. The caches will be represented with the first letter, for example, Caffeine "**C**" and for the run mode **C** for cold and **O** for operational. Due to space consideration, only the low and high scenarios will be presented here.

Low concurrency scenarios Applying the two weights above, we obtain Table 3. Similar values in the same column indicate similar performance. Based on these values, we establish the following ranking. For Weights A we get the ranking 2 and for Weights B we get 3.

$$GO > CO > EO > IO \tag{2}$$
$$IO > GO > CO > EO \tag{3}$$

Note that in this ranking, only the values from the operational mode are taken into account.

Table 3: ICE values for low concurrency scenarios.

Cache	**Mode**	**a**	**b**
C	C	9.18	11.34
C	O	5.80	8.63
E	C	2.29	-5.93
E	O	4.24	1.79
G	C	7.36	11.03
G	O	8.16	10.99
I	C	14.56	-10.65
I	O	1.40	48.35

High concurrency scenarios For high concurrency we applied the same method. Producing the following results in Table 4 and for Weights A we get the ranking 4 and for Weights B we get 5.

$$\text{GO} > \text{EO} > \text{IO} > \text{CO} \tag{4}$$
$$\text{IO} > \text{EO} > \text{GO} > \text{CO} \tag{5}$$

Table 4: ICE values for High concurrency scenarios.

Cache	Mode	a	b
C	C	3.66	0.68
C	O	5.44	0.83
E	C	31.12	19.94
E	O	48.44	27.58
G	C	40.84	4.63
G	O	57.77	6.83
I	C	7.61	5.94
I	O	25.17	38.64

As we can see, different concurrency levels produce different rankings, and different equations yield varying results within the same cache. This implies that different systems and software perform better with certain types of caches, meaning there is no universal answer for which cache is the best.

5 Conclusions and Future Work

As highlighted in Section 2, much of the current literature on caching for Java is outdated, overly theoretical, or limited to narrow and highly contextualized scenarios. This study, although itself limited to a specific type of application, namely data repositories, begins to address these gaps by providing a practical and comprehensive evaluation of multiple caching solutions across different operational conditions. The results show that each cache performs optimally only under specific circumstances. All the tested caches present trade-offs, confirming that no single cache can be considered universally superior. Consequently, selecting an appropriate caching mechanism demands a careful assessment of the system's characteristics, operational context, workload profile, and potential scalability needs, as these factors strongly influence the performance and efficiency of each solution. The work also establishes a reproducible set of metrics and a methodological basis for future comparative studies.

Future work will investigate other cache-related aspects for a wider range of applications, such as eviction algorithms and performance evaluation across different types of cache storage. It will also focus on refining the proposed metric by defining specific weights and by including resource usage and other characteristics in the equations, thereby making them more robust.

Acknowledgments

This work is funded by national funds through FCT – Fundação para a Ciência e a Tecnologia, I.P., and, when applicable, co-funded by EU funds under project/support UID/50008/2025 – Instituto de Telecomunicações with DOI identifier https://doi.org/ 10.54499/UID/50008/2025), and within the scope of projects UIDB/04466/2025 and UIDP/04466/2025, and project 16881 (LISBOA2030-FEDER-00816400; DOI: https://doi.org/10.54499/2023.16583.ICDT)

References

1. G. C. Stierhoff and A. G. Davis, "A history of the IBM Systems Journal," *IEEE Annals of the History of Computing*, vol. 20, no. 1, pp. 29–35, 1998. doi: 10.1109/85.646206.
2. Fernandes, R., Bruyninx, C., Crocker, P., Menut, J., Socquet, A., Vergnolle, M., Avallone, A., Bos, M., Bruni, S., Cardoso, R. & Others A new European service to share GNSS Data and Products. *Annals Of Geophysics*. **65**, DM317 (2022)
3. A. T. Kabakuş, *A Performance Comparison of Java Cache Memory Implementations*, Gümüşhane Üniversitesi Fen Bilimleri Enstitüsü Dergisi, vol. 10, no. 3, pp. 844–852, 2020. doi: 10.17714/gumusfenbil.637942.
4. G. Hasslinger, M. Okhovatzadeh, K. Ntougias, F. Hasslinger, and O. Hohlfeld, *An Overview of Analysis Methods and Evaluation Results for Caching Strategies*, 2023. doi: 10.48550/arXiv.2308.02875.
5. D. Noetzold, A. G. M. Rossetto, L. A. Silva, P. Crocker, and V. R. Q. Leithardt, *JVM optimization: An empirical analysis of JVM configurations for enhanced web application performance*, SoftwareX, vol. 28, p. 101933, 2024. doi: 10.1016/j.softx.2024.101933.
6. S. Kanthed, *Redis vs. Memcached in Microservices Architectures: Caching Strategies*, International Journal of Multidisciplinary Research and Growth Evaluation, vol. 4, no. 3, pp. 1084–1091, May–June 2023. doi: 10.54660/.IJMRGE.2023.4.3.1084-1091.
7. I. Henckel and D. Söderberg, *Database Loading Strategies for an In-Memory Cache in Java*, Master's thesis, Dept. of Computer Science, Lund University, 2022. Available: `https://lup.lub.lu.se/student-papers/record/9080748/file/9080752.pdf`.
8. M. I. Zulfa, R. Hartanto, and A. E. Permanasari, *Caching strategy for Web application – a systematic literature review*, International Journal of Web Information Systems, vol. 16, no. 5, pp. 545–569, 2020. doi: 10.1108/IJWIS-06-2020-0032.
9. Gupta, D., Rani, S. & Ahmed, S. Performance Analysis of Caching Strategies in Information-Centric Networking. *Proceedings Of The 2nd International Conference On Information Management And Machine Intelligence*. pp. 77-85 (2021)
10. Redis Labs, "Redis — The Real-time Data Platform," Redis.io, https://redis.io/, Accessed: Aug. 7, 2025.
11. Google, *Guava: Google Core Libraries for Java*, 2025. [Online]. Available: `https://github.com/google/guava`. Accessed: Oct. 1, 2025.
12. Apache Software Foundation, *Apache Ignite: In-Memory Computing Platform*, 2025. [Online]. Available: `https://ignite.apache.org/`. Accessed: Nov. 12, 2025.
13. Software AG, *Ehcache: Open Source, Standards-Based Cache*, 2025. [Online]. Available: `https://www.ehcache.org/`. Accessed: Sep. 5, 2025.
14. Ben Manes, *Caffeine: A High Performance Caching Library*, 2025. [Online]. Available: `https://github.com/ben-manes/caffeine`. Accessed: Aug. 4, 2025.
15. Vasco Senra, *Repository with data, scripts, graphs, and thesis*, GitHub, 2025. [Online]. Available: `https://github.com/vms2100/Tese`.
16. Amazon, *Implementing Microservices on AWS*, 2025. [Online]. Available: `https://docs.aws.amazon.com/pdfs/whitepapers/latest/microservices-on-aws/microservices-on-aws.pdf`. Accessed: Aug. 25, 2025.
17. Faker, *Faker: Python fake data generator*, 2025. [Online]. Available: `https://pypi.org/project/Faker`.